

From Machine Learning to GPT

The full stack, bottom to top — how every idea builds on the one before it

The Big Picture

Every stage in this roadmap adds exactly ONE new idea on top of everything before it. Nothing gets thrown away — gradient descent from Stage 4 is still the update rule inside GPT; backpropagation from Stage 7 is still how every gradient in a Transformer gets computed. The diagram below shows the full stack, with the persistent threads running up the right side showing exactly which ideas carry all the way to the top.



Six tiers, each adding one new capability on top of the last. The colored threads on the right show which core mechanisms persist across multiple stages — gradient descent and matrix multiplication run through literally everything; backpropagation appears from neural networks onward; learned (rather than fixed) representations appear from embeddings onward.

What's Reused vs. What's New, at Each Step

The stack diagram shows the big picture; this breaks down each individual transition — exactly what carries forward unchanged, and exactly what's genuinely new.

Traditional ML → Neural Networks

REUSED: gradient descent, loss functions, weights as learned parameters

NEW: stacking layers, nonlinear activations, backprop through multiple layers

Neural Networks → CNNs

REUSED: backprop, activations, gradient descent, the training loop shape

NEW: convolutional filters, weight sharing across space, pooling

CNNs → Tokens & Embeddings

REUSED: layers are learned weight matrices, trained the same way

NEW: representing discrete symbols (words) as dense vectors, not spatial grids

Tokens & Embeddings → Self-Attention

REUSED: embeddings as the input representation, linear layers (Q/K/V), backprop

NEW: dynamic, content-based weighting between tokens instead of a fixed pooling/conv pattern

Self-Attention → GPT

REUSED: the entire Transformer block, unchanged, just stacked and scaled up

NEW: next-token training objective, autoregressive generation, sampling strategies

The One-Sentence Version of Each Layer

Traditional ML: find weights that minimize a loss function.

Neural Networks: do that, but stack several layers of it, using the chain rule to train all of them together.

CNNs: do that, but share weights across space so the model doesn't need a separate parameter for every pixel.

Tokens & Embeddings: apply the same "learned weights" idea to representing WORDS instead of pixels.

Self-Attention & Transformers: instead of a fixed way of combining inputs (averaging, sliding a filter), let the model dynamically decide what to combine, for every input, every time.

GPT: stack many of those attention blocks, train on predicting the next token, and generate text one token at a time.

Why This Framing Matters

It's easy to experience Stages 4 through 8 as a sequence of increasingly unfamiliar topics. The more accurate description: it's the SAME small set of ideas — learned weights, a loss function, gradient descent, backpropagation — applied to progressively richer architectures and progressively richer data. A production LLM is not a fundamentally different kind of thing from the linear regression you built in Stage 4. It's the same training loop, wrapped around a much larger, much more sophisticated function — built, piece by piece, exactly the way you built it.